

0.25. Le carré, côté par côté

Le 0.24, plus deux côtés : à gauche, puis en haut. La lettre fait un carré et revient à (0, 0).

1. Ce qu'il faut comprendre

C'est le 0.24, plus deux lignes (et leurs nombres) : les deux côtés du retour.

Ce qui est nouveau ici : rien de plus que des pas **négatifs** sur les deux axes : `-pas` sur X ramène à gauche, `-pas` sur Y ramène en haut. Quatre côtés : **un carré**.

Chaque côté part de là où le précédent s'est arrêté, parce que `x` et `y` sont tenus à jour par la console.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_x>` → [0.17.3. La fonction deplace_x\(\) — avancer sur une ligne](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <deplace_y>` → [0.20.3. La fonction deplace_y\(\) — avancer sur une colonne](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1372 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      66 o ← la console, à l'allumage - la mise en route de la console
deplace_x()    158 o ← main() - une fonction fournie par le langage
deplace_y()    158 o ← main() - une fonction fournie par le langage
main()         135 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases, EcrireNombre - attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() - trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() - efface des cases
EcrireNombre   33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles   18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()      x 4   52 o sur place, dans deplace_x(), deplace_y() + AdresseCase (10 o)
nombre()     x 4   40 o sur place, dans main() + EcrireNombre (33 o)
effacer()    x 2   30 o sur place, dans deplace_x(), deplace_y() + AdresseCase (10 o), EffacerCases (12 o)
image()      x 3    9 o sur place, dans deplace_x(), deplace_y(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
deplace_x() x 2   40 o sur place, dans main()
deplace_y() x 2   40 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_x> // fait avancer une tuile sur sa ligne
#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A
#include <nombre>     // écrit un nombre en chiffres
#include <deplace_y>  // fait avancer une tuile sur sa colonne
```

2. UNE VARIABLE — PAS

```
uint8_t pas = 5; // 5 cases, sur chaque axe
```

3. UNE VARIABLE — X

```
uint8_t x = 0; // la COLONNE de la lettre
```

4. UNE VARIABLE — Y

```
uint8_t y = 0; // la LIGNE de la lettre
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    deplace_x(x, y, ALPHABET[0], pas); // 1. à droite : x vaut 5
    nombre(0, 17, x);
    deplace_y(x, y, ALPHABET[0], pas); // 2. en bas : y vaut 5
    nombre(4, 17, y);

    // Les deux lignes ajoutées : les mêmes, avec -pas.
    deplace_x(x, y, ALPHABET[0], -pas); // 3. à gauche : x revient à 0
    nombre(0, 17, x);
    deplace_y(x, y, ALPHABET[0], -pas); // 4. en haut : y revient à 0
    nombre(4, 17, y);

    // (0,0) → → → → (5,0)
    //   ↑           ↓           le carré : 1 à droite, 2 en bas,
    //   ↑           ↓           3 à gauche, 4 en haut
    // (0,5) ← ← ← ← (5,5)

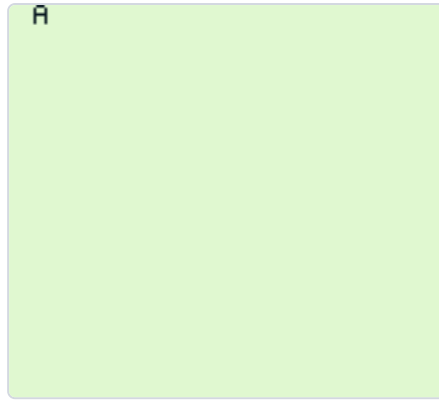
    while (true) {
        image(); // la lettre est revenue en (0, 0)
    }
}
```

3. Ce qui se passe, étape par étape

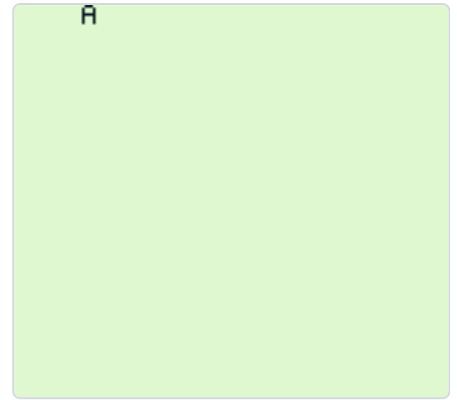
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



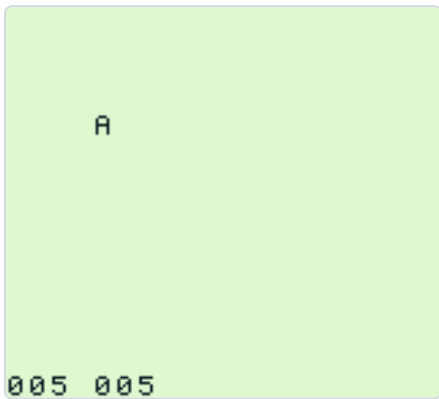
la toute première image — la console vient de démarrer — image 1



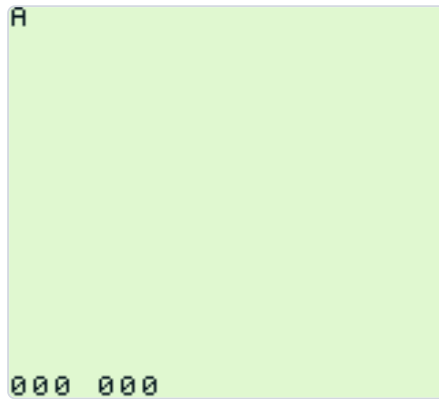
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un A qui fait un carré : 5 cases à droite, 5 en bas, 5 à gauche, 5 en haut. En bas, les nombres changent après chaque côté.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ les coins dans l'ordre : (5, 0), (5, 5), (0, 5), (0, 0) (5,0 5,5 0,5 0,0)

✓ x et y valent 0

✓ 000 et 000 affichés

Ensuite : **0.25.1. Le carré, côté par côté — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++